Lab9 Report: Cloud Image Registry & Distribution

Name：Linjia Guo

Number：202383910005

1. Experiment Overview

This experiment focuses on cloud image registry and container image distribution. In previous labs, the web application was mainly built and tested in a local or development environment. In this lab, the application image was pushed to GitHub Container Registry, so that it could be stored, versioned, distributed, and redeployed from the cloud.

The experiment simulates a basic cloud engineering workflow. First, a Docker image was built from a static web application. Then, the image was tagged according to the naming rules of GitHub Container Registry. After that, the image was pushed to the cloud registry and verified through GitHub Packages. A second version of the application was also created to observe Docker’ s layered storage mechanism. Finally, local Docker data was deleted to simulate a clean environment, and the application was redeployed directly from the cloud registry.

---

2. Experimental Purpose

The main purposes of this experiment are as follows:

1. To understand how cloud image registries are used to store and distribute container images.
2. To practice secure authentication with GitHub Container Registry by using a Personal Access Token instead of a GitHub password.
3. To learn how Docker image tags are used to manage different application versions, such as v1.0 and v2.0.
4. To push Docker images to GitHub Container Registry and verify them in GitHub Packages.
5. To observe Docker layer caching during image pushing and understand why repeated uploads can be faster.
6. To simulate disaster recovery by deleting local images and containers, then redeploying the application directly from the cloud.

7. To compare standard Nginx images with Alpine-based images and analyze the effect of image size on cloud deployment.

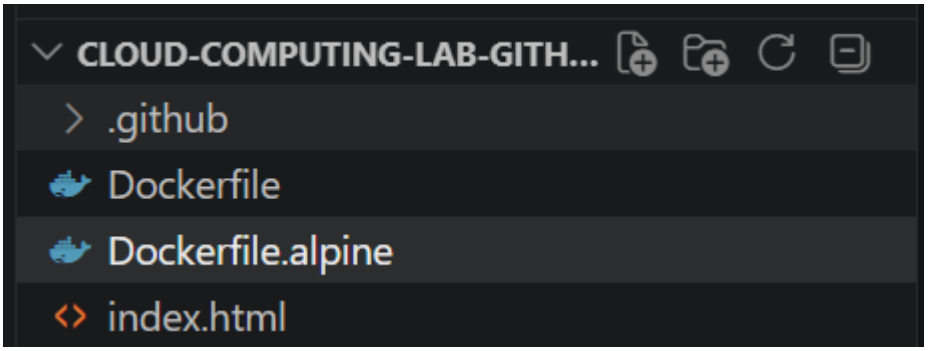
---

3. Experimental Environment

The experiment was carried out in GitHub Codespaces. The project repository contained the required application files, including `index.html` and `Dockerfile`. Docker was already available in the Codespaces terminal, so the image could be built, tagged, pushed, pulled, and executed directly in the cloud development environment.

The main tools used in this experiment were:

Item	Description
Cloud Development Environment	GitHub Codespaces
Image Registry	GitHub Container Registry
Container Tool	Docker
Web Server	Nginx
Source Code Platform	GitHub
Application Type	Static web page



---

4. Experimental Process

4.1 Secure Authentication with GitHub Container Registry

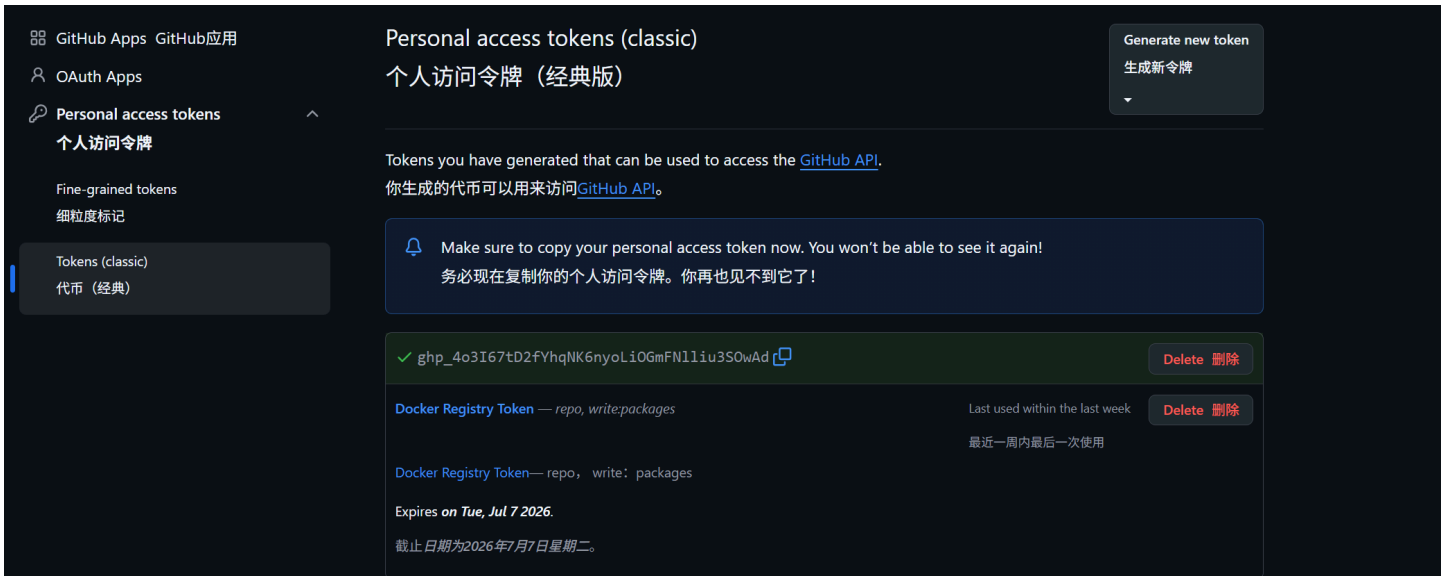
Before pushing Docker images to GitHub Container Registry, authentication was required. Instead of using the GitHub account password directly in the terminal, I generated a Personal

Access Token in GitHub. This method is safer because the token can be limited to specific permissions and revoked when necessary.

The token was generated through:

`GitHub Settings → Developer settings → Personal access tokens → Tokens (classic)`

The required permission was selected as `write:packages`. This permission allows Docker images to be pushed to GitHub Container Registry. The related package reading permission was also included automatically.



After generating the token, I logged in to GitHub Container Registry in the Codespaces terminal. The token was passed to Docker through standard input, which avoids typing the token directly as a visible password.

Code block

```
export CR_PAT=YOUR_COPIED_TOKEN
echo $CR_PAT | docker login ghcr.io -u YOUR_GITHUB_USERNAME --password-stdin
```

After running the command, the terminal displayed `Login Succeeded`, which proved that the authentication was successful.

```
@yaox45882-pro ➜ /workspaces/cloud-computing-lab-github (main) $ export CR_PAT=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
@yaox45882-pro ➜ /workspaces/cloud-computing-lab-github (main) $ echo $CR_PAT | docker login ghcr.io -u yaox45882-pro --password-stdin

WARNING! Your credentials are stored unencrypted in '/home/codespace/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
```

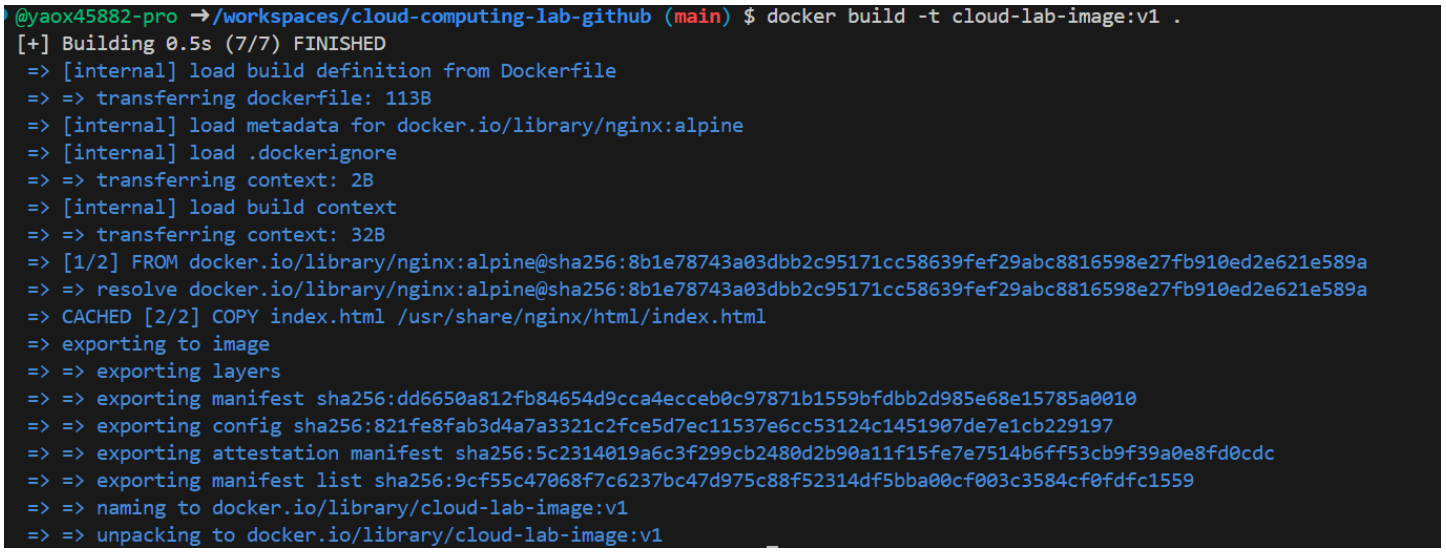
4.2 Build the First Docker Image

After authentication, I built the first version of the application image. The image was named `cloud-lab-image:v1`. This image packaged the static web page together with the Nginx runtime environment.

Code block

```
docker build -t cloud-lab-image:v1 .
```

This step created a local Docker image. The image could be used locally, but it still needed to be tagged correctly before being pushed to the cloud registry.



4.3 Tag the Image for GitHub Container Registry

Docker images must follow the naming format required by the target registry. For GitHub Container Registry, the image name should include `ghcr.io`, the GitHub username, the package name, and the version tag.

The local image was tagged as follows:

Code block

```
docker tag cloud-lab-image:v1 ghcr.io/yaox45882-pro/cloud-app:v1.0
docker images
```

The tag `v1.0` indicates the first released version of the application. By using version tags, different iterations of the same application can be managed clearly.

```
@yaox45882-pro ➜ /workspaces/cloud-computing-lab-github (main) $ docker tag cloud-lab-image:v1 ghcr.io/yaox45882-pro/cloud-app:v1.0
@yaox45882-pro ➜ /workspaces/cloud-computing-lab-github (main) $ docker images

IMAGE                                      ID             DISK USAGE   CONTENT SIZE   EXTRA
cloud-app:alpine                           c7de31859eab       93.3MB         26.1MB
cloud-app:standard                         a362c2378dd0       92.7MB           26MB   U
cloud-lab-image:v1                         9cf55c47068f       92.7MB           26MB   U
cloud-lab-image:v2                         7b90d6b7004d       92.7MB           26MB   U
ghcr.io/yaox45882-pro/cloud-app:v1.0       9cf55c47068f       92.7MB           26MB   U
ghcr.io/yaox45882-pro/cloud-app:v2.0       7b90d6b7004d       92.7MB           26MB   U
nginx:alpine                               8b1e78743a03       93.6MB           27MB
nginx:latest                               5aca99593157        241MB           66MB
```

4.4 Push Version 1.0 to GitHub Container Registry

After the image was tagged, I pushed it to GitHub Container Registry.

Code block

```
docker push ghcr.io/yaox45882-pro/cloud-app:v1.0
```

During this process, Docker uploaded the image layers to GitHub’ s container registry infrastructure. After the push was completed, the image became available in the cloud and could be pulled from other environments.

```
@yaox45882-pro ➜ /workspaces/cloud-computing-lab-github (main) $ docker push ghcr.io/yaox45882-pro/cloud-app:v1.0
The push refers to repository [ghcr.io/yaox45882-pro/cloud-app]
a5008f4a4b25: Layer already exists
abaae85d1626: Layer already exists
43f834d60d8a: Layer already exists
de1b677d8c00: Layer already exists
94d083cf706a: Layer already exists
e654dbbbb9e1: Layer already exists
fbaed3f7fcbe: Layer already exists
b32957e56639: Pushed
6a87c5000886: Layer already exists
6a0ac1617861: Layer already exists
v1.0: digest: sha256:9cf55c47068f7c6237bc47d975c88f52314df5bba00cf003c3584cf0fdfc1559 size: 856
```

To verify whether the upload was successful, I opened my GitHub profile and checked the Packages section. The package named `cloud-app` appeared under my GitHub account, which confirmed that the image had been uploaded successfully.

4.5 Modify the Application for Version 2.0

To simulate a real application update, I modified the `index.html` file. A visible change was added to the web page, such as changing the displayed text to show that this was Version 2.0.

For example, the page title or main heading could be changed to:

Code block

```
Welcome to Version 2.0
```

This modification made it possible to distinguish the updated version from the original version when the application was redeployed.

```
</head>

<body>
    <div class="container">
        <h1>Cloud-Native Deployment Success! It's Lab 9!!!</h1>
        <h2>Welcome to Version 2.0</h2>

        <p>This website was automatically deployed using <b>GitHub Actions</b>.</p>

        <div class="status">● Site Status: Live</div>
```

4.6 Rebuild and Retag the Version 2.0 Image

After modifying the web page, I rebuilt the Docker image and tagged it as version 2.0.

Code block

```
docker build -t cloud-lab-image:v2 .
docker tag cloud-lab-image:v2 ghcr.io/yaox45882-pro/cloud-app:v2.0
docker images
```

The new local image was named `cloud-lab-image:v2`, and the cloud registry tag was `cloud-app:v2.0`. This shows how Docker tags can be used to manage different application versions.

```
● @yaox45882-pro →/workspaces/cloud-computing-lab-github (main) $ docker tag cloud-lab-image:v2 ghcr.io/yaox45882-pro/cloud-app:v2.0
● @yaox45882-pro →/workspaces/cloud-computing-lab-github (main) $ docker images

IMAGE                                    ID             DISK USAGE   CONTENT SIZE   EXTRA
cloud-app:alpine                         c7de31859eab       93.3MB         26.1MB
cloud-app:standard                       a362c2378dd0       92.7MB           26MB   U
cloud-lab-image:v1                       9cf55c47068f       92.7MB           26MB   U
cloud-lab-image:v2                       428dfc5af237       92.7MB           26MB   U
ghcr.io/yaox45882-pro/cloud-app:v1.0     9cf55c47068f       92.7MB           26MB   U
ghcr.io/yaox45882-pro/cloud-app:v2.0     428dfc5af237       92.7MB           26MB   U
nginx:alpine                             8b1e78743a03       93.6MB           27MB
nginx:latest                             5aca99593157        241MB           66MB
```

4.7 Push Version 2.0 and Observe Layer Caching

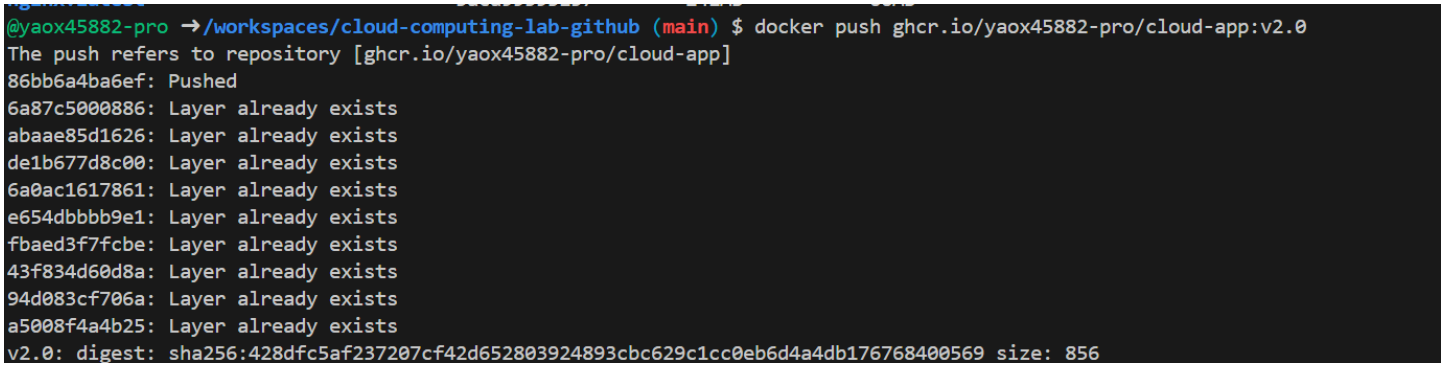
Next, I pushed the Version 2.0 image to GitHub Container Registry.

Code block

```
docker push ghcr.io/yaox45882-pro/cloud-app:v2.0
```

During the push process, Docker displayed messages such as `Layer already exists`. This means that many image layers were already stored in the remote registry from the previous version. Since only the application file had changed, Docker did not need to upload every layer again.

This demonstrates Docker’ s layered storage mechanism. The unchanged layers can be reused, while only the modified layers need to be uploaded. As a result, pushing Version 2.0 is usually faster than pushing Version 1.0.



4.8 Disaster Recovery: Delete Local Docker Data

To simulate a clean server or disaster recovery environment, I removed local Docker images, containers, networks, and build cache.

Code block

```
docker system prune -a
```

This command deletes local Docker resources. After this step, the local machine no longer relied on previously built images. Therefore, if the application could still be deployed successfully, it would prove that the cloud registry could act as an independent image distribution source.

```
@yaox45882-pro ➜/workspaces/cloud-computing-lab-github (main) $ docker system prune -a
deleted: sha256:40c71f75d204ae142f45dd5b3573d986f0950cb51056580622ef97d0866d9c2a

Deleted build cache objects:
wokcsthwtzg1mpi9xp0mv65gn
0x5w5hteq13v2e74mig7m9xmj
tm7yebc8b24amguvr0gphmuhy
mezyg6s8dz5kvtnrytmzk3ib9
kr2z8xy8pdzyiq4jaadcygac6
wakksolcmxlj3d48a8g5zial0
o9skxbj22mlc88fu59bhntzae
judkr2u8xq9pbqaariunbnw3b
v7ay60py6tg5dhu22hkxohmlv
q1elwzx7l2ixz19rlramjc7fk
dw8t57z3rgbd3u5gssn0tavou
4fxb7p2i6omt7scwaabwjl1oo
l1ub2fy4tywv3mt5na2z5afik
5no6sgjiqvscdcf7ls2t8d83c

Total reclaimed space: 93.27MB
```

4.9 Pull and Run the Image from the Cloud Registry

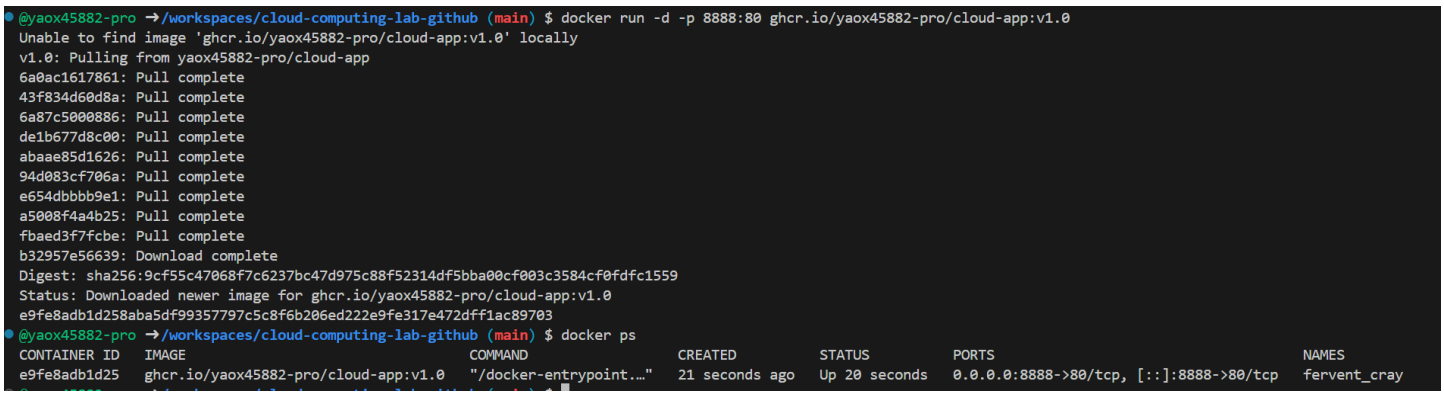
After cleaning local Docker data, I deployed the application directly from GitHub Container Registry.

Code block

```
docker run -d -p 8888:80 ghcr.io/yaox45882-pro/cloud-app:v1.0
docker ps
```

Because the local image had been deleted, Docker automatically pulled the image from the cloud registry and then started the container. The container mapped port 80 inside the container to port 8888 in the Codespaces environment.

This step proved that the application image was no longer dependent on the original local development environment or source code files.



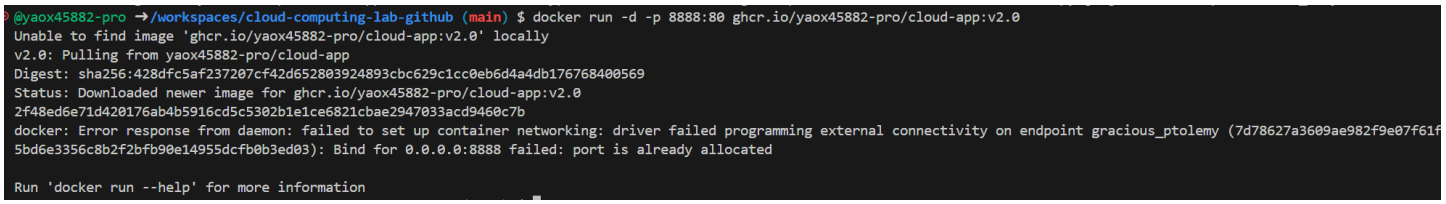
4.10 Port Conflict Problem and Solution

When I tried to run another container using the same host port 8888, Docker returned an error because the port was already occupied.

Code block

```
docker run -d -p 8888:80 ghcr.io/yaox45882-pro/cloud-app:v2.0
```

The error occurred because one host port can only be bound to one running container at the same time. Since the previous container was already using port 8888, the new container could not start with the same port mapping.



To solve this problem, I first checked the running container:

Code block

```
docker ps
```

Then I stopped the container that was occupying port 8888:

Code block

```
docker stop CONTAINER_ID
```

After the old container was stopped, I ran the Version 2.0 container again:

Code block

```
docker run -d -p 8888:80 ghcr.io/yaox45882-pro/cloud-app:v2.0
```

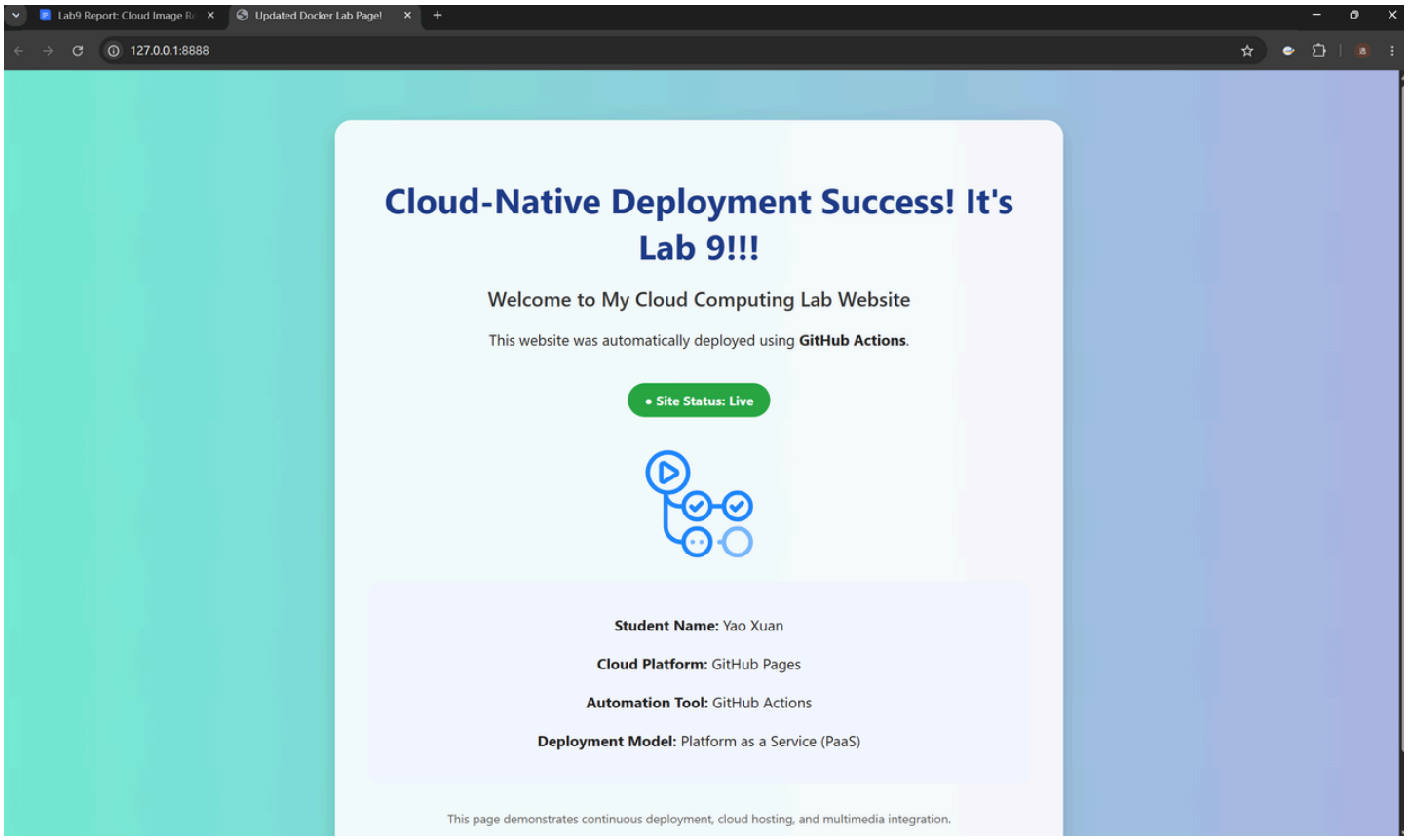
This time, the container started successfully.

```
Run 'docker run --help' for more information
@yaox45882-pro ➜/workspaces/cloud-computing-lab-github (main) $
@yaox45882-pro ➜/workspaces/cloud-computing-lab-github (main) $ docker ps
CONTAINER ID   IMAGE                                  COMMAND                  CREATED              STATUS              PORTS                                       NAMES
e9fe8adb1d25   ghcr.io/yaox45882-pro/cloud-app:v1.0   "/docker-entrypoint.…"   About a minute ago   Up About a minute   0.0.0.0:8888->80/tcp, [::]:8888->80/tcp     fervent_cray
@yaox45882-pro ➜/workspaces/cloud-computing-lab-github (main) $ docker stop IMAGE
Error response from daemon: No such container: IMAGE
@yaox45882-pro ➜/workspaces/cloud-computing-lab-github (main) $ docker stop e9fe8adb1d25
e9fe8adb1d25
@yaox45882-pro ➜/workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8888:80 ghcr.io/yaox45882-pro/cloud-app:v2.0
0b8be690ff42ed1909f4d252a31315bcfdd17bdf8dcb0c49fa2371c4d8358236
@yaox45882-pro ➜/workspaces/cloud-computing-lab-github (main) $ docker ps
CONTAINER ID   IMAGE                                  COMMAND                  CREATED         STATUS         PORTS                                       NAMES
0b8be690ff42   ghcr.io/yaox45882-pro/cloud-app:v2.0   "/docker-entrypoint.…"   7 seconds ago   Up 7 seconds   0.0.0.0:8888->80/tcp, [::]:8888->80/tcp     confident_shirley
@yaox45882-pro ➜/workspaces/cloud-computing-lab-github (main) $
```

4.11 Verify the Application in Browser

After the Version 2.0 container was started, I opened the forwarded port 8888 in the browser. The web page displayed the updated content, which confirmed that Version 2.0 had been deployed correctly.

This verification step was important because it showed that the image pulled from the cloud registry could run successfully and provide the expected service.



4.12 Image Optimization: Standard Image vs. Alpine Image

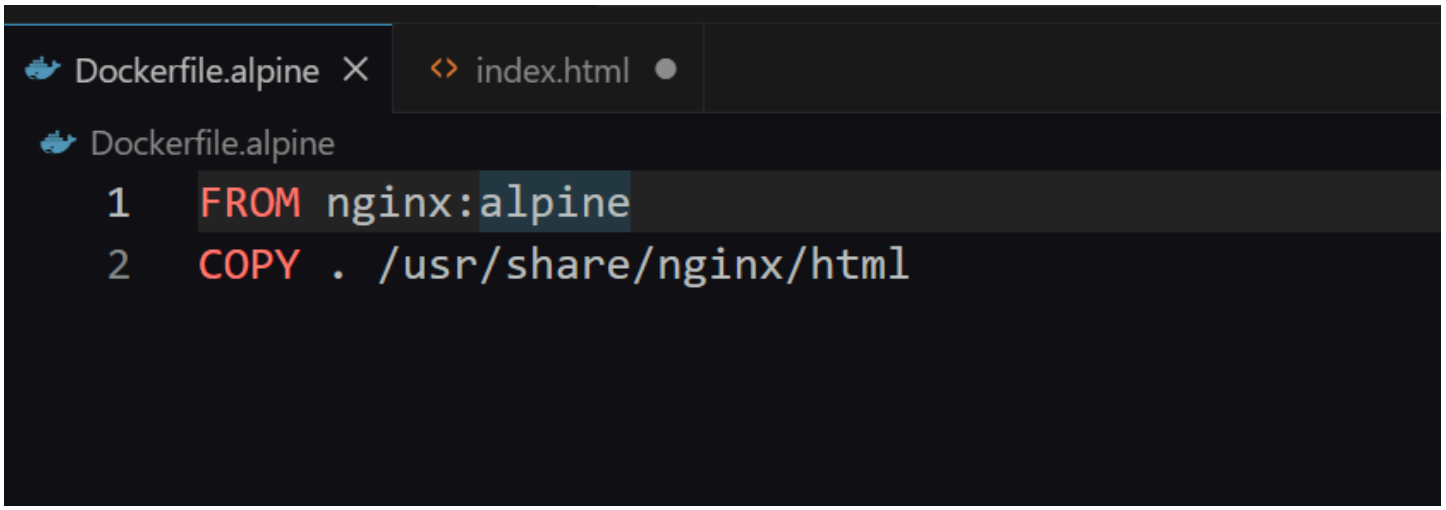
In cloud computing, image size affects storage cost, network transmission time, and deployment speed. Therefore, I compared a standard Nginx-based image with an Alpine-based

image.

First, I created a new file named `Dockerfile.alpine`:

```
Code block

1   FROM nginx:alpine
2   COPY . /usr/share/nginx/html
```

Alpine Linux is a lightweight Linux distribution. It contains fewer default packages than standard Linux distributions, so images based on Alpine are usually smaller.



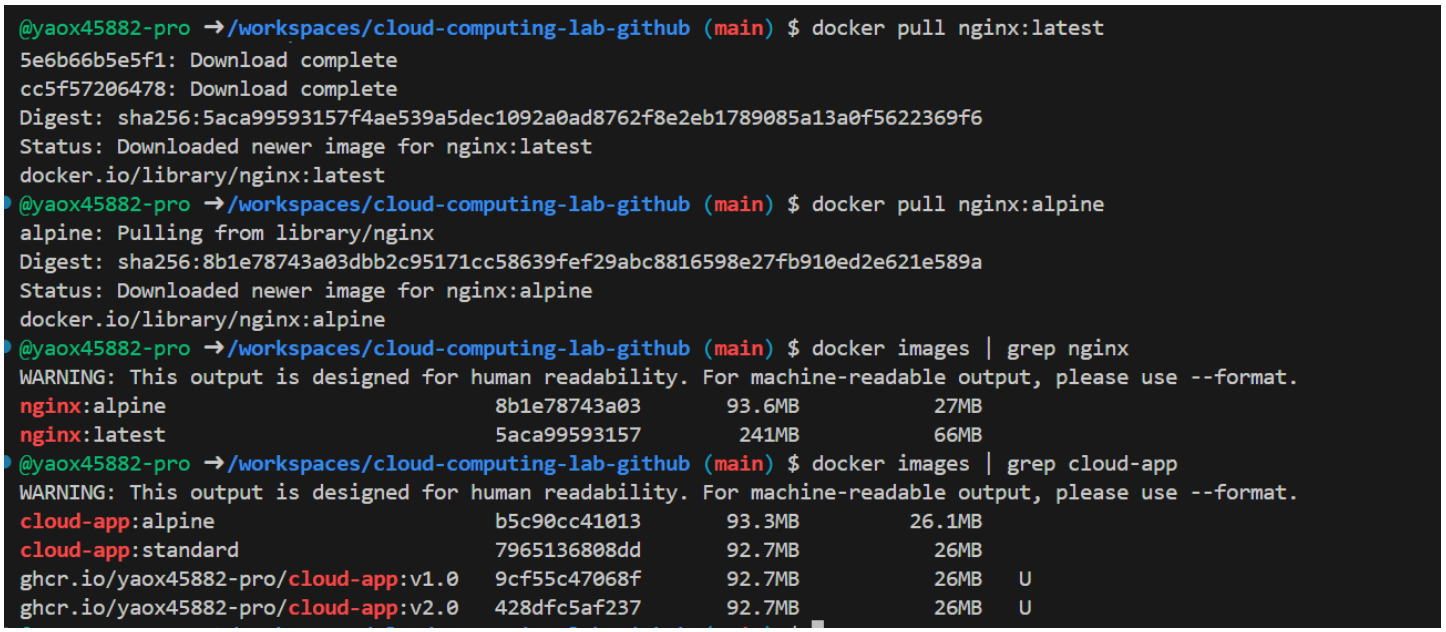
Then I built both versions of the image:

```
Code block

1   docker build -t cloud-app:standard -f Dockerfile .
2   docker build -t cloud-app:alpine -f Dockerfile.alpine .
```

To compare the base image sizes, I also pulled `nginx:latest` and `nginx:alpine`.

```
Code block

1   docker pull nginx:latest
2   docker pull nginx:alpine
3   docker images | grep nginx
4   docker images | grep cloud-app
```

The image size comparison showed that the Alpine-based image was much smaller than the standard Nginx image.



---

5. Experimental Results and Analysis

The main experimental results are summarized below.

Step	Operation	Result
Secure authentication	Logged in to GHCR using PAT	Login succeeded
Build v1 image	Built cloud-lab-image:v1	Image created successfully
Tag v1.0	Tagged image for GHCR	Cloud-ready tag created
Push v1.0	Pushed image to GHCR	Image uploaded successfully
Verify package	Checked GitHub Packages	cloud-app package appeared
Modify application	Updated index.html	Version 2.0 change added
Push v2.0	Pushed updated image	Layer caching observed
Disaster recovery	Deleted local Docker data	Local environment cleaned
Remote deployment	Pulled image from GHCR	Application redeployed
Port conflict	Reused port 8888	Error observed and solved
Image optimization	Compared standard and Alpine images	Image sizes compared

The image size comparison can be recorded as follows. The exact values should be filled in according to the screenshots from the terminal.

Image	Size	Observation
nginx:latest	____241__ MB	Standard Nginx image
nginx:alpine	___93.6___ MB	Lightweight Alpine-based image
cloud-app:standard	____92.7__ MB	Application image based on standard Dockerfile
cloud-app:alpine	___93.3___ MB	Application image based on Alpine Dockerfile

From the result, the Alpine-based image is generally smaller. A smaller image is useful in cloud environments because it reduces storage consumption and can improve deployment speed, especially when containers need to be started frequently.

---

6. Discussion

6.1 Why Version 2.0 Was Pushed Faster

When pushing Version 2.0, Docker reported that several layers already existed. This happened because Docker images are composed of multiple layers. If the base image and most files remain unchanged, Docker can reuse the existing layers stored in the remote registry.

Compared with Version 1.0, only the modified web page layer changed in Version 2.0. Therefore, Docker only needed to upload the changed layer instead of uploading the entire image again. This is the reason why the Version 2.0 push was faster.

Layer caching is very important in cloud computing because it saves network bandwidth, reduces upload time, and improves the efficiency of continuous deployment.

6.2 Why PAT Is More Secure Than Password Authentication

Using a Personal Access Token is safer than using a GitHub account password in the terminal. A PAT can be configured with limited permissions. For example, in this experiment, the token only needed package-related permissions.

In addition, a PAT can be revoked independently. If the token is accidentally exposed, it can be deleted from GitHub settings without changing the account password. However, if the real

password is exposed, the whole GitHub account may be at risk.

Therefore, using a PAT follows better security practices for cloud development and automation.

6.3 Why Alpine Images Are Common in Production

Alpine images are widely used in production microservice environments because they are small and lightweight. A smaller image has several advantages:

1. It requires less storage space in the image registry.
2. It can be pulled faster during deployment.
3. It can reduce cold start time when containers are started.
4. It reduces unnecessary packages in the runtime environment.

However, Alpine images may sometimes require extra configuration if an application depends on libraries that are not included by default. For simple static web applications such as this experiment, Alpine is a good choice because the runtime requirements are minimal.

---

7. Problems Encountered and Solutions

Problem	Cause	Solution
Token should not be exposed	PAT can be used like a password	Store it carefully and regenerate it if exposed
Docker login warning appeared	Credentials may be stored unencrypted in Codespaces	This is acceptable in the lab environment if login succeeds
Version 2.0 page did not update immediately	Browser cache or old container still running	Refresh browser, stop old container, and run the new image
Port 8888 was already allocated	Another container was already using the same host port	Stop the old container or use a different host port
Local image disappeared after cleanup	docker system prune -a removed local images	Pull the image again from GitHub Container Registry
Alpine image size needed comparison	Different base images have different sizes	Use docker images to check and record exact sizes

---

8. Git Submission

At the end of the experiment, I saved the updated files to GitHub. This included the modified `index.html` file and the newly created `Dockerfile.alpine`.

The following commands were used:

Code block

```
git add .
git commit -m "Finished Lab 03"
git push
```

This step ensured that the source files and experiment changes were stored permanently in the remote GitHub repository.

Screenshot 18: Git commit and successful push to GitHub.

---

9. Experimental Summary

In this experiment, I completed a full cloud image registry and distribution workflow. I used a Personal Access Token to securely log in to GitHub Container Registry, built a Docker image for a static Nginx web application, tagged the image according to GHCR naming rules, and pushed it to GitHub Packages.

I also created a second version of the application and pushed it as `v2.0`. During this process, Docker layer caching was observed. The message `Layer already exists` showed that Docker reused unchanged layers and only uploaded the modified part of the image. This helped me understand why container image updates can be efficient in cloud environments.

The disaster recovery test was one of the most meaningful parts of the experiment. After deleting local Docker images and containers, I was still able to pull and run the application from GitHub Container Registry. This proved that the container image stored in the cloud registry could be used as an independent deployment artifact.

The port conflict step also helped me understand Docker networking. A host port can only be occupied by one running container at a time. To solve the problem, I stopped the old container before running the new one, or I could choose another host port.

Finally, the image optimization part showed why lightweight images such as Alpine are important in cloud computing. Smaller images can reduce storage costs, shorten pull time, and improve deployment efficiency. Overall, this experiment connected several important cloud computing concepts, including secure authentication, image versioning, cloud registry distribution, disaster recovery, layer caching, port management, and image optimization.